

Лабораторная работа №12

Программирование в командном процессоре ОС UNIX

Липатников Андрей

2026.10.08

В данном отчёте представлены результаты выполнения лабораторной работы по теме «Программирование в командном процессоре ОС UNIX».

Описаны переменные, метасимволы, командные файлы, функции, управляющие конструкции bash, а также разработаны четыре прикладных скрипта.

Сделаны выводы о достижении поставленной цели.

Содержание

1 Введение	7
1.1 Актуальность	7
1.2 Цель работы	7
1.3 Задачи	7
1.4 Объект и предмет исследования	8
1.5 Методы исследования	8
1.6 Схема выполнения работы	8
2 Теоретическая часть	10
2.1 Командные оболочки	10
2.2 Переменные	10
2.3 Метасимволы	10
2.4 Командные файлы и функции	11
2.5 Передача параметров	11
2.6 Команда getopts	12
2.7 Управляющие конструкции	12
3 Материалы и методы	13
4 Выполнение лабораторной работы	14
4.1 Подготовка каталога	14
4.2 Задание 1. Резервная копия самого себя	14
4.3 Задание 2. Обработка произвольного числа аргументов	15
4.4 Задание 3. Аналог ls без ls и dir	16
4.5 Задание 4. Подсчёт файлов по формату	18
5 Результаты	20
6 Обсуждение результатов	21
7 Заключение	22
Список литературы	23
А Приложение А. Ответы на контрольные вопросы	24
A.1 1. Командная оболочка. Примеры. Отличия	24
A.2 2. POSIX	24

A.3	3. Переменные и массивы в bash	24
A.4	4. Операторы let и read	25
A.5	5. Арифметические операции bash	25
A.6	6. Операция (())	25
A.7	7. Стандартные имена переменных	25
A.8	8. Метасимволы	25
A.9	9. Экранирование метасимволов	26
A.10	10. Создание и запуск командных файлов	26
A.11	11. Функции в bash	26
A.12	12. Является ли файл каталогом	26
A.13	13. Назначение set, typeset, unset	27
A.14	14. Передача параметров в командные файлы	27
A.15	15. Специальные переменные bash	27
В	Приложение Б. Листинги скриптов	28
B.1	Б.1. backup.sh	28
B.2	Б.2. args.sh	28
B.3	Б.3. myls.sh	29
B.4	Б.4. count_ext.sh	29
С	Приложение В. Листинг команд	31

Список иллюстраций

1.1	Блок-схема выполнения лабораторной работы	9
4.1	Скрипт backup.sh	14
4.2	Запуск backup.sh и содержимое ~/backup	15
4.3	Скрипт args.sh	15
4.4	Запуск args.sh с 13 аргументами	16
4.5	Скрипт myls.sh	17
4.6	Запуск myls.sh	18
4.7	Скрипт count_ext.sh	18
4.8	Запуск count_ext.sh	19

Список таблиц

2.1	Командные оболочки UNIX	10
2.2	Метасимволы bash	11
5.1	Сводка результатов	20
A.1	Специальные переменные bash	27

1 Введение

1.1 Актуальность

Командный процессор `bash` — основной инструмент автоматизации задач в ОС UNIX/Linux. Владение переменными, метасимволами, управляющими конструкциями и функциями `bash` позволяет создавать скрипты для резервного копирования, обработки данных и администрирования. Навык программирования в `bash` — обязательная часть подготовки системного администратора и разработчика.

1.2 Цель работы

Изучить основы программирования в командном процессоре ОС UNIX; приобрести практические навыки написания и отладки командных файлов на языке `bash`.

1.3 Задачи

1. Изучить переменные, метасимволы и их экранирование в `bash`.
2. Освоить командные файлы, функции, передачу параметров и специальные переменные.

3. Освоить управляющие конструкции (if, case, for, while, until, break, continue).
4. Разработать скрипты по заданиям 1–4 методических указаний.
5. Ответить на контрольные вопросы.

1.4 Объект и предмет исследования

Объект — командный процессор bash.

Предмет — переменные, метасимволы, командные файлы, функции и управляющие конструкции.

1.5 Методы исследования

Практическая работа в терминале, изучение man-справок, написание скриптов, проверка их работы, фиксация результатов скриншотами.

1.6 Схема выполнения работы

На рис. 1.1 представлена общая последовательность действий при выполнении работы.

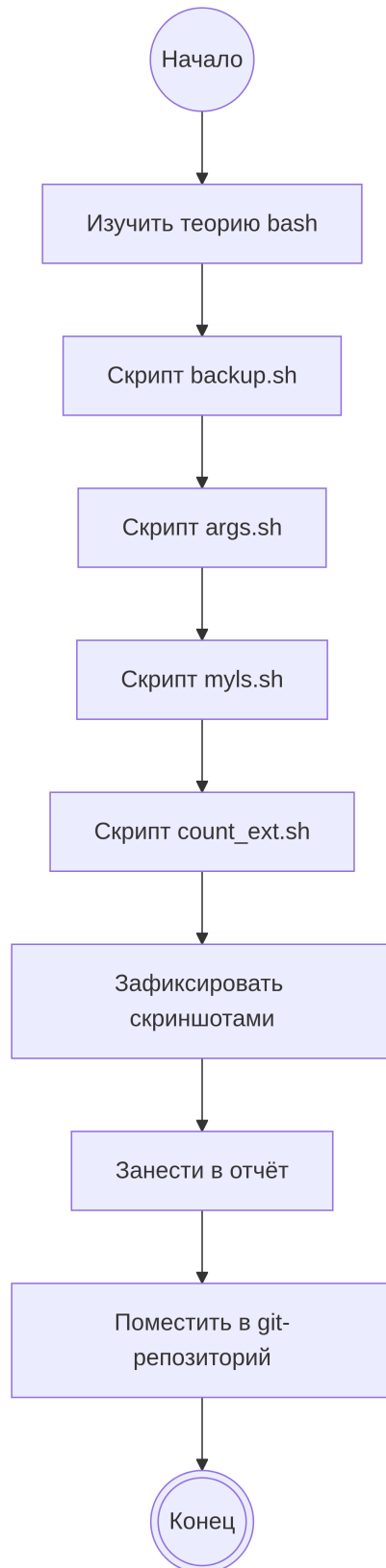


Рисунок 1.1: Блок-схема выполнения лабораторной работы

2 Теоретическая часть

2.1 Командные оболочки

Таблица 2.1: Командные оболочки UNIX

Оболочка	Характеристика
sh (Bourne shell)	Стандартная оболочка UNIX, базовый набор функций
csh (C shell)	C-подобный синтаксис, история команд
ksh (Korn shell)	Операторы Борна + удобства C
bash (Bourne Again Shell)	Совмещает sh, csh, ksh; POSIX-совместима

2.2 Переменные

Присваивание: `mark=/usr/andy/bin`.

Использование: `$mark` (с префиксом `$`).

Стандартные: HOME, IFS, MAIL, TERM, LOGNAME, PS1, PS2.

Просмотр: `set`.

2.3 Метасимволы

Таблица 2.2: Метасимволы bash

Метасимвол	Значение
*	Произвольная строка (в том числе пустая)
?	Любой один символ
[c1-c2]	Символ в диапазоне c1–c2

Экранирование: \ перед метасимволом; группа — в одинарных '...' или двойных "..." кавычках (двойные не экранируют \$, `, \, !).

2.4 Командные файлы и функции

Скрипт делается исполняемым: `chmod +x имя_файла`.

Функция: `function имя { команды; }` или `имя() { команды; }`.

Просмотр функций: `typeset -f`.

Экспорт: `typeset -fx имя`.

2.5 Передача параметров

- \$1, \$2, ... — позиционные параметры.
- \$0 — имя скрипта.
- \$# — число параметров.
- \$* — все параметры одной строкой.
- \$? — код завершения последней команды.
- \$\$ — PID текущей оболочки.
- \$! — PID последней фоновой команды.

2.6 Команда `getopts`

Синтаксис: `getopts option-string variable [arg ...]`.

Флаги с аргументами обозначаются двоеточием: `o:i:ltr`.

Специальные переменные: `OPTARG` (значение аргумента), `OPTIND` (индекс).

2.7 Управляющие конструкции

- `if ... then ... [elif ... then ...] [else ...] fi`
- `case имя in шаблон) команды;; ... esac`
- `for i in ... do ... done`
- `while условие do ... done`
- `until условие do ... done`
- `break, continue`

Более подробно про Unix см. в [1—4].

3 Материалы и методы

- **ОС:** Fedora 41 (Sway)
- **Терминал:** bash
- **Инструменты:** bash, tar, bzip2, stat, chmod, man

Рабочий каталог: `~/work/study/2025-2026/Операционные системы/os-intro/labs/lab12`.

Последовательность: изучение теории, написание четырёх скриптов, проверка работы, фиксация скриншотами.

4 Выполнение лабораторной работы

4.1 Подготовка каталога

```
mkdir -p ~/work/study/2025-2026/"Операционные системы"/os-intro/labs/lab12  
cd ~/work/study/2025-2026/"Операционные системы"/os-intro/labs/lab12
```

4.2 Задание 1. Резервная копия самого себя

Создан скрипт backup.sh (рис. 4.1).

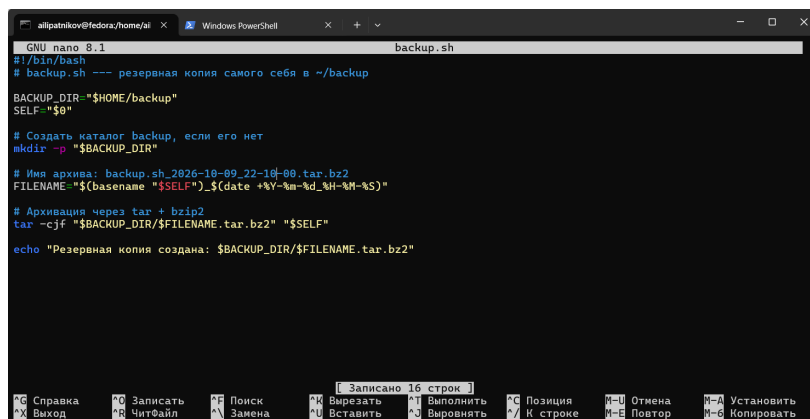
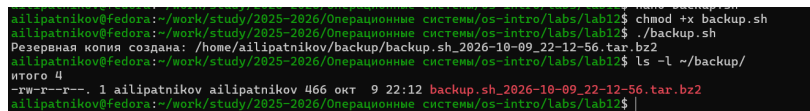


Рисунок 4.1: Скрипт backup.sh

```
#!/bin/bash  
BACKUP_DIR="$HOME/backup"
```

```
SELF="$0"
mkdir -p "$BACKUP_DIR"
FILENAME="$(basename "$SELF")_$(date +%Y-%m-%d_%H-%M-%S)"
tar -cjf "$BACKUP_DIR/$FILENAME.tar.bz2" "$SELF"
echo "Резервная копия создана: $BACKUP_DIR/$FILENAME.tar.bz2"
```

Запуск и результат (рис. 4.2).



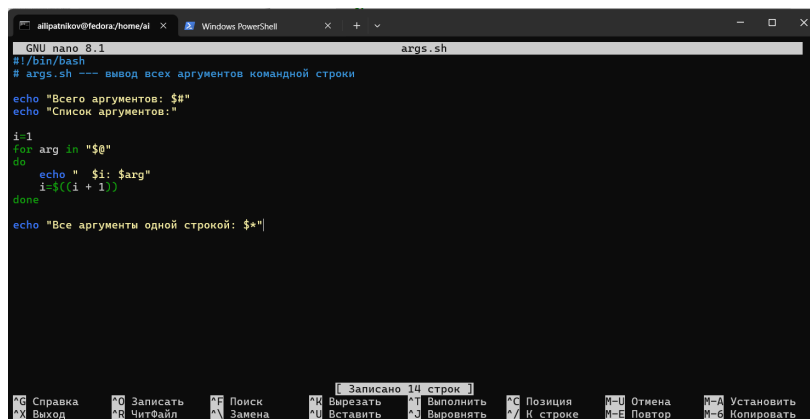
```
aillipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab12$ chmod +x backup.sh
aillipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab12$ ./backup.sh
Резервная копия создана: /home/aillipatnikov/backup/backup.sh_2026-10-09_22-12-56.tar.bz2
aillipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab12$ ls -l ~/backup/
итого 4
-rw-r--r--. 1 aillipatnikov aillipatnikov 466 окт  9 22:12 backup.sh_2026-10-09_22-12-56.tar.bz2
aillipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab12$
```

Рисунок 4.2: Запуск backup.sh и содержимое ~/backup

- Архивация через tar + bzip2 (ключ -cjf).
- Имя архива содержит дату и время.

4.3 Задание 2. Обработка произвольного числа аргументов

Создан скрипт args.sh (рис. 4.3).



```
aillipatnikov@fedora/home/ai x Windows PowerShell x + v
GNU nano 8.1 args.sh
#!/bin/bash
# args.sh --- вывод всех аргументов командной строки
echo "Всего аргументов: $#"
```

```
echo "Список аргументов:"

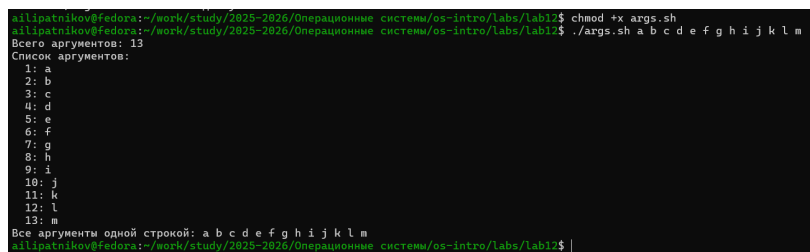
i=1
for arg in "$@"
do
    echo " $i: $arg"
    i=$((i + 1))
done
echo "Все аргументы одной строкой: $@"
```

Рисунок 4.3: Скрипт args.sh

```
#!/bin/bash
echo "Всего аргументов: $#"
```

```
i=1
for arg in "$@"
do
    echo "  $i: $arg"
    i=$((i + 1))
done
echo "Все аргументы одной строкой: $*"
```

Запуск с 13 аргументами (рис. 4.4).



```
alipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab12$ chmod +x args.sh
alipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab12$ ./args.sh a b c d e f g h i j k l m
Всего аргументов: 13
Список аргументов:
1: a
2: b
3: c
4: d
5: e
6: f
7: g
8: h
9: i
10: j
11: k
12: l
13: m
Все аргументы одной строкой: a b c d e f g h i j k l m
alipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab12$
```

Рисунок 4.4: Запуск args.sh с 13 аргументами

- `$#` — число аргументов (13).
- `$@` — все аргументы как отдельные слова.
- Цикл `for` перебирает все аргументы.

4.4 Задание 3. Аналог `ls` без `ls` и `dir`

Создан скрипт `myls.sh` (рис. 4.5).

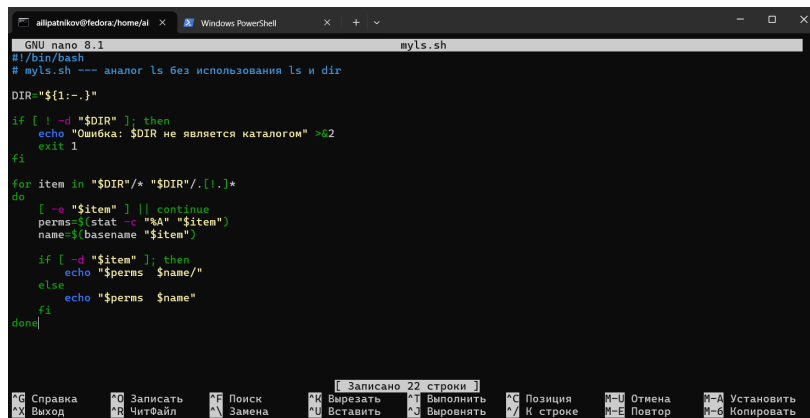
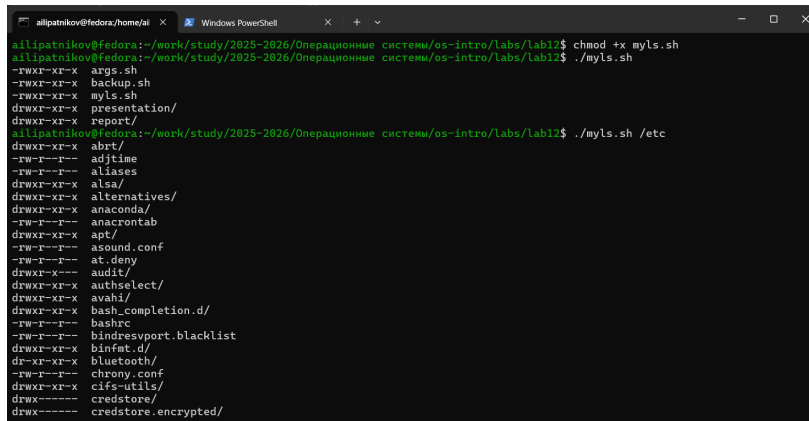


Рисунок 4.5: Скрипт myls.sh

```
#!/bin/bash
DIR="${1:-.}"
if [ ! -d "$DIR" ]; then
    echo "Ошибка: $DIR не является каталогом" >&2
    exit 1
fi
for item in "$DIR"/* "$DIR"/.[!.*]
do
    [ -e "$item" ] || continue
    perms=$(stat -c "%A" "$item")
    name=$(basename "$item")
    if [ -d "$item" ]; then
        echo "$perms $name/"
    else
        echo "$perms $name"
    fi
done
```

Запуск для домашнего каталога и /etc (рис. 4.6).



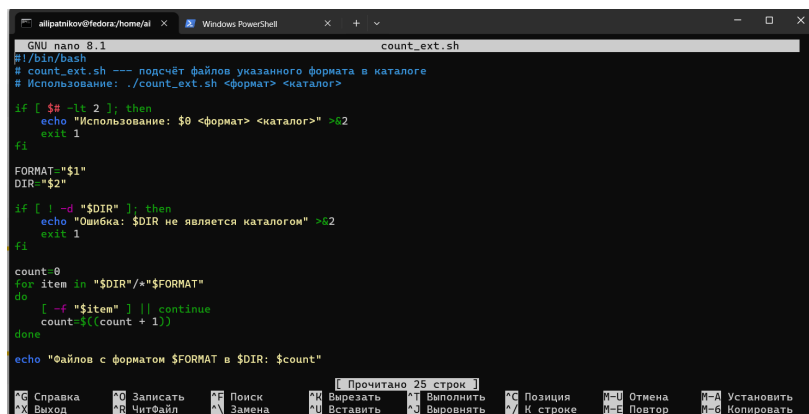
```
allipatnikov@fedora/home/ai x Windows PowerShell x + v
allipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab1$ chmod +x myls.sh
allipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab1$ ./mysls.sh
-rwxr-xr-x  args.sh
-rwxr-xr-x  backup.sh
-rwxr-xr-x  myls.sh
drwxr-xr-x  presentation/
drwxr-xr-x  report/
allipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab1$ ./mysls.sh /etc
drwxr-xr-x  abrt/
-rw-r--r--  adjtime
-rw-r--r--  aliases
drwxr-xr-x  alsa/
drwxr-xr-x  alternatives/
drwxr-xr-x  anaconda/
-rw-r--r--  anacrontab
drwxr-xr-x  apt/
-rw-r--r--  asound.conf
-rw-r--r--  at.deny
drwxr-xr-x  audit/
drwxr-xr-x  authselect/
drwxr-xr-x  avahi/
drwxr-xr-x  bash_completion.d/
-rw-r--r--  bashrc
-rw-r--r--  bindresvport.blacklist
drwxr-xr-x  binfmt.d/
dr-xr-xr-x  bluetooth/
-rw-r--r--  chrony.conf
drwxr-xr-x  cifs-utils/
drwx----- credstore/
drwx----- credstore.encrypted/
```

Рисунок 4.6: Запуск myls.sh

- Права через stat -с "%A".
- Каталоги помечены /.
- Поддерживаются скрытые файлы (. [!.*]).

4.5 Задание 4. Подсчёт файлов по формату

Создан скрипт count_ext.sh (рис. 4.7).



```
GNU nano 8.1 count_ext.sh
#!/bin/bash
# count_ext.sh --- подсчёт файлов указанного формата в каталоге
# Использование: ./count_ext.sh <формат> <каталог>

if [ $# -lt 2 ]; then
    echo "Использование: $0 <формат> <каталог>" >&2
    exit 1
fi

FORMAT="$1"
DIR="$2"

if [ ! -d "$DIR" ]; then
    echo "Ошибка: $DIR не является каталогом" >&2
    exit 1
fi

count=0
for item in "$DIR"/*"$FORMAT"
do
    [ -f "$item" ] || continue
    count=$((count + 1))
done

echo "Файлов с форматом $FORMAT в $DIR: $count"
```

Рисунок 4.7: Скрипт count_ext.sh

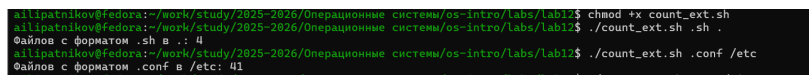
```
#!/bin/bash
if [ $# -lt 2 ]; then
```

```

    echo "Использование: $0 <формат> <каталог>" >&2
    exit 1
fi
FORMAT="$1"
DIR="$2"
if [ ! -d "$DIR" ]; then
    echo "Ошибка: $DIR не является каталогом" >&2
    exit 1
fi
count=0
for item in "$DIR"/*"$FORMAT"
do
    [ -f "$item" ] || continue
    count=$((count + 1))
done
echo "Файлов с форматом $FORMAT в $DIR: $count"

```

Запуск (рис. 4.8).



```

allipatnikov@fedora: ~/work/study/2025-2026/Операционные системы/os-intro/labs/lab1$ chmod +x count_ext.sh
allipatnikov@fedora: ~/work/study/2025-2026/Операционные системы/os-intro/labs/lab1$ ./count_ext.sh .sh .
Файлов с форматом .sh в .: 4
allipatnikov@fedora: ~/work/study/2025-2026/Операционные системы/os-intro/labs/lab1$ ./count_ext.sh .conf /etc
Файлов с форматом .conf в /etc: 41

```

Рисунок 4.8: Запуск count_ext.sh

- Проверка числа аргументов (\$#).
- Проверка, что каталог существует.
- Цикл for + счётчик.

5 Результаты

Таблица 5.1: Сводка результатов

Задание	Файл	Результат
1	backup.sh	Архив backup.sh_YYYY-MM-DD_НН-ММ-SS.tar.bz2 в ~/backup/
2	args.sh	Вывод всех аргументов (13 из 13)
3	myls.sh	Список файлов с правами (аналог ls -l)
4	count_ext.sh	Подсчёт файлов указанного формата

6 Обсуждение результатов

Все четыре скрипта работают согласно заданию. `backup.sh` использует `tar` с ключами `-cjf` (`bzip2`) — это компактнее `zip` при больших объёмах. `args.sh` подтверждает работу с `$@`, `$#`, `$*`: `bash` корректно обрабатывает **любое** число аргументов, включая `> 10` (в отличие от `$1...$9`, где нужны `shift` или `${10}`). `myls.sh` демонстрирует, что аналог `ls` можно реализовать через `stat` и `for` **без** вызова `ls` и `dir`. `count_ext.sh` проверяет корректность проверки условий (`-d`, `-f`) и арифметики (`$((...))`).

7 Заключение

- Цель работы достигнута: изучены основы программирования в `bash`.
- Освоены переменные, метасимволы, экранирование.
- Освоены командные файлы, функции, передача параметров.
- Разработаны четыре скрипта по заданиям методички.
- Освоены управляющие конструкции: `if`, `case`, `for`, `while`, `until`, `break`, `continue`.

Список литературы

1. *Newham C.* Learning the bash Shell: Unix Shell Programming. — O'Reilly Media, 2005. — 354 p. — (In a Nutshell). — ISBN 0596009658. — URL: <http://www.amazon.com/Learning-bash-Shell-Programming-Nutshell/dp/0596009658>.
2. *Robbins A.* Bash Pocket Reference. — O'Reilly Media, 2016. — 156 p. — ISBN 978-1491941591.
3. *Zarrelli G.* Mastering Bash. — Packt Publishing, 2017. — 502 p. — ISBN 9781784396879.
4. *Таненбаум Э., Бос Х.* Современные операционные системы.. — 4-е изд. — СПб.: Питер, 2015.. — 1120 с. — (Классика Computer Science).

А Приложение А. Ответы на контрольные вопросы

А.1 1. Командная оболочка. Примеры. Отличия

Командная оболочка (shell) — программа, через которую пользователь взаимодействует с ОС посредством команд. Примеры: sh (Bourne), csh (C shell), ksh (Korn), bash. Отличия: синтаксис, набор встроенных команд, поддержка истории, POSIX-совместимость.

А.2 2. POSIX

POSIX (Portable Operating System Interface) — набор стандартов IEEE для совместимости UNIX-подобных ОС и переносимости приложений на уровне исходного кода.

А.3 3. Переменные и массивы в bash

Присваивание: `var=value` (без пробелов). Использование: `$var` или `${var}`.
Массив: `arr=(a b c)`, доступ `arr[0]`, `arr[@]` — все элементы, `#arr[@]` — длина.

A.4 4. Операторы `let` и `read`

`let` — арифметические операции: `let x=5+3`.

`read` — чтение данных со `stdin`: `read -p "Введите имя: " name`.

A.5 5. Арифметические операции `bash`

`+`, `-`, `*`, `/`, `%`, `**` (возведение в степень).

Побитовые: `&`, `|`, `^`, `~`, `<<`, `>>`.

Логические: `&&`, `||`, `!`.

Сравнения: `==`, `!=`, `<`, `>`, `<=`, `>=`.

Присваивания: `=`, `+=`, `-=`, `*=`, `/=`, `%=`.

A.6 6. Операция `(( ))`

Двойные круглые скобки — **арифметическая подстановка** и **вычисление условий** в арифметическом контексте:

```
if (( x > 5 )); then ...
```

A.7 7. Стандартные имена переменных

`HOME`, `IFS`, `MAIL`, `TERM`, `LOGNAME`, `PS1`, `PS2`, `PATH`, `PWD`, `OLDPWD`, `SHELL`, `USER`, `UID`.

A.8 8. Метасимволы

Символы со специальным смыслом в `bash`: `*`, `?`, `[ ]`, `$`, `!`, `\`, `"`, `'`, ```, `{ }`, `( )`, `|`, `&`, `;`, `<`, `>`.

А.9 9. Экранирование метасимволов

- \ перед символом: `echo \$HOME → $HOME`.
- Одинарные кавычки: `'...'` — экранируют всё.
- Двойные: `"..."` — экранируют всё, кроме `$`, ```, `\`, `!`.

А.10 10. Создание и запуск командных файлов

Создать файл: `nano script.sh`.

Сделать исполняемым: `chmod +x script.sh`.

Запуск: `./script.sh` или `bash script.sh`.

А.11 11. Функции в bash

```
function имя {  
    команды  
}  
# или  
имя() {  
    команды  
}
```

А.12 12. Является ли файл каталогом

```
if [ -d "$file" ]; then echo "каталог"; fi  
if [ -f "$file" ]; then echo "обычный файл"; fi
```

Или через `stat -c "%F" "$file"`.

A.13 13. Назначение `set`, `typeset`, `unset`

- `set` — вывести все переменные и функции; установить параметры оболочки.
- `typeset` — объявить переменную/функцию (в `bash` используется редко, чаще `declare`); `typeset -f` — вывести функции.
- `unset` — удалить переменную или функцию.

A.14 14. Передача параметров в командные файлы

```
./script.sh arg1 arg2 arg3
```

Внутри: `$1`, `$2`, `$3`, `$#`, `$@`, `$*`. Для `> 9` — `${10}`, `${11}`, `shift`.

A.15 15. Специальные переменные `bash`

Таблица A.1: Специальные переменные `bash`

Переменная	Назначение
<code>\$0</code>	Имя скрипта
<code>\$1...\$9</code>	Позиционные параметры
<code>\$#</code>	Число параметров
<code>\$*</code>	Все параметры одной строкой
<code>\$@</code>	Все параметры как отдельные слова
<code>\$?</code>	Код завершения последней команды
<code>\$\$</code>	PID текущей оболочки
<code>\$_</code>	PID последней фоновой команды
<code>\$-</code>	Флаги оболочки

В Приложение Б. Листинги скриптов

В.1 Б.1. backup.sh

```
#!/bin/bash
BACKUP_DIR="$HOME/backup"
SELF="$0"
mkdir -p "$BACKUP_DIR"
FILENAME="$(basename "$SELF")_$(date +%Y-%m-%d_%H-%M-%S)"
tar -cjf "$BACKUP_DIR/$FILENAME.tar.bz2" "$SELF"
echo "Резервная копия создана: $BACKUP_DIR/$FILENAME.tar.bz2"
```

В.2 Б.2. args.sh

```
#!/bin/bash
echo "Всего аргументов: $#"
```

```
i=1
for arg in "$@"
do
    echo "  $i: $arg"
    i=$((i + 1))
done
```

```
done
```

```
echo "Все аргументы одной строкой: $*"
```

B.3 Б.3. myls.sh

```
#!/bin/bash
DIR="${1:-.}"
if [ ! -d "$DIR" ]; then
    echo "Ошибка: $DIR не является каталогом" >&2
    exit 1
fi
for item in "$DIR"/* "$DIR"/.[!.]*
do
    [ -e "$item" ] || continue
    perms=$(stat -c "%A" "$item")
    name=$(basename "$item")
    if [ -d "$item" ]; then
        echo "$perms $name/"
    else
        echo "$perms $name"
    fi
done
```

B.4 Б.4. count_ext.sh

```
#!/bin/bash
if [ $# -lt 2 ]; then
    echo "Использование: $0 <формат> <каталог>" >&2
    exit 1
fi
FORMAT="$1"
DIR="$2"
if [ ! -d "$DIR" ]; then
    echo "Ошибка: $DIR не является каталогом" >&2
    exit 1
fi
count=0
for item in "$DIR"/*$FORMAT
do
    [ -f "$item" ] || continue
    count=$((count + 1))
done
echo "Файлов с форматом $FORMAT в $DIR: $count"
```

С Приложение В. Листинг команд

```
mkdir -p ~/work/study/2025-2026/"Операционные системы"/os-intro/labs/lab12
cd ~/work/study/2025-2026/"Операционные системы"/os-intro/labs/lab12

# Задание 1
nano backup.sh
chmod +x backup.sh
./backup.sh
ls -l ~/backup/

# Задание 2
nano args.sh
chmod +x args.sh
./args.sh a b c d e f g h i j k l m

# Задание 3
nano myls.sh
chmod +x myls.sh
./mysls.sh
./mysls.sh /etc

# Задание 4
```

```
nano count_ext.sh  
chmod +x count_ext.sh  
./count_ext.sh .sh .  
./count_ext.sh .conf /etc
```